

The CTMT Covariance:

Seepage-Induced Observational Covariance in $\theta \in \mathbb{R}^d$: Stabilization–Jacobian Pipeline

Rada Matěj

Email: MatejRada@email.cz

This work is licensed under a Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 License.

Contents

1	Introduction	2
2	Local covariance decomposition	2
3	Canonical seepage support induced by the Jacobian	3
4	Canonical estimators in finite samples	4
5	Weighted Fisher and GLS geometry	4
6	Two-pass inversion pipeline	5
6.1	Synthetic generator	5
6.2	Diagnostics and acceptance criteria	5
6.3	Complete Python test script	6
7	What is proved, what is tested, and what is not claimed	11
8	Suggested results framing	12
9	Falsifier and failure modes	12
10	Discussion	12

Abstract

We study when an effective observational covariance may be treated as an induced object rather than as an externally imposed statistical prior. In a local stabilization–Jacobian pipeline, measurements are modeled as resolved transport through a differentiable forward map plus an unresolved latent component and sensor noise. Under a CTMT admissibility hypothesis, unresolved seepage transport is required to be observationally orthogonal to resolved transport, i.e. supported in the Jacobian-null observable complement. We prove that this hypothesis uniquely fixes the support projector of admissible unresolved covariance. If residual second moments and sensor-noise covariance are known exactly, then the admissible seepage covariance itself is unique. In finite samples, this yields a canonical projected estimator obtained by restricting the empirical residual covariance to the Jacobian-null support and regularizing only within that support. We implement a full synthetic magnetostatic test battery on a Biot–Savart coil model with anisotropic sensor noise, heavy-tailed bursts, and null-supported latent perturbations. The numerical protocol separates three claims: support uniqueness, exact covariance uniqueness under known second moments, and finite-sample recoverability. In supportive runs, the projected estimator recovers the dominant unresolved covariance structure, shows low leakage into resolved directions, improves parameter inversion relative to identity weighting, and preserves coarse-graining admissibility. We also state explicit falsifiers and non-claims. The main conclusion is deliberately limited: the Jacobian

does not determine the full unresolved spectrum by itself, but it does determine the admissible support geometry, and exact covariance uniqueness follows once residual second moments and sensor noise are fixed.

1 Introduction

Local inverse problems are typically posed through a forward map $f(\theta)$ from parameter space to observable space, together with a local linearization

$$y \approx y_0 + J \Delta\theta, \quad y_0 := f(\theta_0), \quad J := \left. \frac{\partial f}{\partial \theta} \right|_{\theta_0}. \quad (1)$$

Weighted least squares or Fisher-based inference then proceeds after choosing an observational covariance model C . In standard treatments, this covariance is either assumed known, estimated directly in observable coordinates, or regularized by general shrinkage principles. The present note asks a narrower structural question: when may an effective covariance be regarded as induced by the local forward geometry itself?

In the CTMT setting, one does not assume that the resolved parameterization exhausts all local transport. Instead, one allows a latent unresolved component acting in observable space,

$$y = y_0 + J \Delta\theta + u + \varepsilon, \quad (2)$$

where u denotes unresolved seepage transport and ε ordinary measurement noise.

The key claim studied here is not that the Jacobian determines the full covariance spectrum by itself. Rather, the claim is that under an admissibility hypothesis on unresolved transport, the Jacobian determines the *support geometry* of admissible unresolved covariance, and that exact covariance uniqueness follows once residual second moments and sensor noise are fixed.

This formulation is materially stronger and more defensible than the informal slogan that J “derives” C unqualifiedly. It separates three levels:

- (i) **Support uniqueness:** the Jacobian uniquely fixes the admissible unresolved support projector;
- (ii) **Exact covariance uniqueness:** if residual second moments and sensor noise are known exactly, then the admissible unresolved covariance is unique;
- (iii) **Finite-sample recoverability:** in real or synthetic data, one estimates a projected and regularized approximation of that exact object.

2 Local covariance decomposition

Proposition 1 (Local seepage covariance). *Assume the local model (2). Define the residual after removing resolved transport by*

$$r := y - y_0 - J \Delta\theta = u + \varepsilon.$$

Then

$$C_{\text{eff}} := \text{Cov}(r) = C_{\text{seep}} + \Sigma_{\varepsilon}, \quad C_{\text{seep}} := \text{Cov}(u), \quad (3)$$

where $\Sigma_{\varepsilon} := \text{Cov}(\varepsilon)$. The cross terms vanish by assumption.

Remark 1. Equation (3) is elementary covariance algebra. Its CTMT content lies not in the decomposition itself, but in the hypothesis that admissible unresolved covariance is geometrically constrained rather than free to occupy arbitrary observable directions.

3 Canonical seepage support induced by the Jacobian

Let

$$J = U\Sigma V^\top$$

be a compact singular value decomposition with rank r , and let

$$\Pi_r := U_r U_r^\top, \quad \Pi_s := I - \Pi_r. \quad (4)$$

Then Π_r projects onto the resolved observable image $\mathcal{R}(J)$, whereas Π_s projects onto $\ker(J^\top) = \mathcal{R}(J)^\perp$.

The CTMT admissibility postulate is:

$$J^\top u = 0. \quad (5)$$

Theorem 1 (Canonical support of admissible seepage covariance). *Assume every admissible unresolved perturbation $u \in \mathbb{R}^m$ satisfies (5). Then:*

- (1) $u \in \ker(J^\top)$ almost surely;
- (2) the seepage covariance satisfies

$$\text{range}(C_{\text{seep}}) \subseteq \ker(J^\top);$$

- (3) equivalently,

$$C_{\text{seep}} = \Pi_s C_{\text{seep}} \Pi_s;$$

- (4) for any orthonormal basis U_s of $\ker(J^\top)$, one may write $u = U_s \zeta$ for some latent coefficient vector ζ .

Corollary 1 (Uniqueness of the Jacobian-induced support projector). *The orthogonal projector onto admissible unresolved support is uniquely determined by the Jacobian as*

$$\Pi_s = I - JJ^+, \quad (6)$$

where J^+ is the Moore–Penrose pseudoinverse.

Theorem 2 (Exact uniqueness of admissible seepage covariance). *Assume:*

- (i) the residual covariance $C_r := \text{Cov}(r)$ is known exactly;
- (ii) the sensor-noise covariance Σ_ε is known exactly;
- (iii) seepage is observationally orthogonal, so $C_{\text{seep}} = \Pi_s C_{\text{seep}} \Pi_s$.

Then $C_r = C_{\text{seep}} + \Sigma_\varepsilon$ contains a unique admissible seepage covariance:

$$C_{\text{seep}} = \Pi_s (C_r - \Sigma_\varepsilon) \Pi_s, \quad (7)$$

provided the consistency conditions

$$\Pi_s (C_r - \Sigma_\varepsilon) \Pi_r = 0, \quad \Pi_s (C_r - \Sigma_\varepsilon) \Pi_s \succeq 0 \quad (8)$$

hold.

Remark 2. This theorem is the precise sense in which covariance may be called *induced*. The Jacobian alone does not determine the full spectrum. What it determines is the admissible support projector; once exact residual second moments and exact sensor noise are supplied, no further freedom remains.

4 Canonical estimators in finite samples

Given windowed residuals

$$r_w := y_w - f(\hat{\theta}_w),$$

their sample covariance is

$$\hat{S} = \frac{1}{W-1} \sum_{w=1}^W (r_w - \bar{r})(r_w - \bar{r})^\top, \quad \bar{r} := \frac{1}{W} \sum_{w=1}^W r_w. \quad (9)$$

Projecting into the canonical seepage support:

$$\hat{S}_\perp := \Pi_s \hat{S} \Pi_s. \quad (10)$$

Proposition 2 (Canonical projection as the nearest admissible covariance). *Among all symmetric matrices M satisfying $M = \Pi_s M \Pi_s$, the matrix $\hat{S}_\perp$ uniquely minimizes*

$$\|\hat{S} - M\|_F^2.$$

If $\hat{S} \succeq 0$, then $\hat{S}_\perp \succeq 0$.

To stabilize finite-sample estimation, regularize only inside the admissible support:

$$\hat{\Lambda}_s = (1 - \lambda) \hat{S}_s + \lambda \text{diag}(\hat{S}_s) + \rho I, \quad \lambda \in [0, 1], \rho > 0, \quad (11)$$

and reconstruct

$$\hat{C}_{\text{seep}} = U_s \hat{\Lambda}_s U_s^\top. \quad (12)$$

With known or estimated sensor-noise covariance $\hat{\Sigma}_\varepsilon$,

$$\hat{C}_{\text{eff}} = \hat{C}_{\text{seep}} + \hat{\Sigma}_\varepsilon. \quad (13)$$

Corollary 2 (Unique isotropic fallback on the seepage support). *Fix a total seepage power $\tau > 0$. Among all positive semidefinite C satisfying $C = \Pi_s C \Pi_s$ and $\text{Tr}(C) = \tau$, the unique isotropic fallback is*

$$C_{\text{seep}}^* = \frac{\tau}{m-r} \Pi_s. \quad (14)$$

5 Weighted Fisher and GLS geometry

Given a positive-definite covariance model C , the weighted local Fisher matrix is

$$F(\theta_0; C) = J^\top C^{-1} J. \quad (15)$$

The corresponding generalized least-squares update is

$$\Delta \hat{\theta} = (J^\top C^{-1} J)^+ J^\top C^{-1} (y - y_0). \quad (16)$$

Under the canonical support theorem,

$$C_{\text{eff}} = U_s \Lambda_s U_s^\top + \Sigma_\varepsilon, \quad (17)$$

whence

$$F(\theta_0; C_{\text{eff}}) = J^\top (U_s \Lambda_s U_s^\top + \Sigma_\varepsilon)^{-1} J. \quad (18)$$

6 Two-pass inversion pipeline

The inversion used below has two passes.

Pass 1: identity-weighted baseline. Set $C_I = \sigma^2 I$, solve (16), collect residuals.

Pass 2: canonical seepage weighting. Compute U_s or Π_s , estimate $\hat{C}_{\text{eff}}$, recompute GLS update:

$$\Delta \hat{\theta}_w^{(\text{CTMT})} = (J_w^\top \hat{C}_{\text{eff},w}^{-1} J_w)^+ J_w^\top \hat{C}_{\text{eff},w}^{-1} (y_w - y_{0,w}). \quad (19)$$

We validate the construction on a synthetic magnetostatic coil model. The forward map is a Biot–Savart line integral for a circular loop of radius $R = 5$ cm, sampled by 60 axial and 40 radial sensors. The observable is the vertical field component B_z . The parameter vector is

$$\theta = (I, dx, dy), \quad (20)$$

and the Jacobian is computed by finite differences around the local anchor

$$\theta_0 = (4.0, 0, 0), \quad \theta_{\text{true}} = (4.0, 1.2 \times 10^{-3}, -7.5 \times 10^{-4}). \quad (21)$$

6.1 Synthetic generator

Each window is generated as

$$y_w = f(\theta_{\text{true}}) + u_w + \varepsilon_w, \quad (22)$$

where:

- $\varepsilon_w \sim \mathcal{N}(0, \Sigma_\varepsilon)$ is anisotropic sensor noise, optionally contaminated by occasional heavy-tailed bursts;
- $u_w \sim \mathcal{N}(0, C_{\text{seep},\text{true}})$ is latent seepage transport supported in $\ker(J^\top)$, with

$$C_{\text{seep},\text{true}} = U_s \Lambda_{\text{true}} U_s^\top,$$

for some fixed diagonal spectrum $\Lambda_{\text{true}} \succeq 0$ on the Jacobian-null basis U_s .

6.2 Diagnostics and acceptance criteria

Each Monte Carlo run over W windows reports:

- Inversion gain:** RMSE and window-to-window spread under identity weighting versus canonical seepage weighting;
- Covariance recovery:** relative Frobenius error and top- k eigenspace overlap between $\hat{C}_{\text{seep}}$ and $C_{\text{seep},\text{true}}$;
- Support leakage:** the ratio

$$\ell := \frac{\|\Pi_r \hat{C}_{\text{seep}} \Pi_r\|_F + \|\Pi_r \hat{C}_{\text{seep}} \Pi_s\|_F}{\|\hat{C}_{\text{seep}}\|_F}, \quad (23)$$

which should remain near numerical tolerance if the support claim holds;

- Exact uniqueness sanity check:** when the true second moments are inserted, the exact uniqueness formula (7) should recover $C_{\text{seep},\text{true}}$ up to machine precision;
- Monotonicity:** the coarse-grained Fisher matrix must satisfy $F_{\text{cg}} \preceq F$.

A run is counted as supportive only if all of the following hold:

$$\text{RMSE}(\hat{C}) < \text{RMSE}(I), \quad \text{overlap}(E_k(\hat{C}_{\text{seep}}), E_k(C_{\text{seep},\text{true}})) \gg \text{random baseline}, \quad \ell \ll 1, \quad F_{\text{cg}} \preceq F. \quad (24)$$

6.3 Complete Python test script

The following script implements the full synthetic battery in auditable form.

```
1 import numpy as np
2 import numpy.linalg as npl
3
4 mu0 = 4e-7 * np.pi
5
6
7 def loop_polyline(radius=0.05, nseg=400, center=(0.0, 0.0, 0.0)):
8     phi = np.linspace(0, 2*np.pi, nseg, endpoint=False)
9     x = radius * np.cos(phi) + center[0]
10    y = radius * np.sin(phi) + center[1]
11    z = np.zeros_like(phi) + center[2]
12    return np.c_[x, y, z]
13
14
15 def axial_and_radial_sensors(nz=60, zmax=0.06, nr=40, rmax=0.08):
16    z_axis = np.linspace(-zmax, zmax, nz)
17    axis_pts = np.c_[np.zeros(nz), np.zeros(nz), z_axis]
18    r_line = np.linspace(0.0, rmax, nr)
19    rad_pts = np.c_[r_line, np.zeros(nr), np.zeros(nr)]
20    return np.vstack([axis_pts, rad_pts])
21
22
23 def biot_savart_line(sensors, polyline, current=1.0):
24    Ns = sensors.shape[0]
25    P0 = polyline
26    P1 = np.roll(polyline, -1, axis=0)
27    dL = P1 - P0
28    B = np.zeros((Ns, 3))
29    for i in range(Ns):
30        r = sensors[i][None, :] - P0
31        r_norm = np.linalg.norm(r, axis=1)
32        r_hat = r / r_norm[:, None]
33        cross = np.cross(dL, r_hat)
34        contrib = mu0 * current * cross / (4 * np.pi * r_norm[:,
35            None]**2)
36        B[i, :] = np.sum(contrib, axis=0)
37    return B
38
39 def biot_savart_multi(sensors, conductors, currents):
40    B = np.zeros((sensors.shape[0], 3))
41    for poly, I in zip(conductors, currents):
42        B += biot_savart_line(sensors, poly, current=I)
43    return B
44
45
46 def forward_Bz(theta, sensors, conductors):
47    I = theta[0]
48    dx, dy = theta[1], theta[2]
49    shifted = [poly + np.array([dx, dy, 0.0]) for poly in conductors]
50    return biot_savart_multi(sensors, shifted, [I] *
51        len(conductors))[:, 2]
52
53 def jacobian_fd(theta0, sensors, conductors, h_frac=1e-6):
```

```

54     base = forward_Bz(theta0, sensors, conductors)
55     d = len(theta0)
56     J = np.zeros((base.size, d))
57     for j in range(d):
58         t1 = theta0.copy()
59         step = h_frac * max(1.0, abs(theta0[j]))
60         t1[j] += step
61         pert = forward_Bz(t1, sensors, conductors)
62         J[:, j] = (pert - base) / step
63     return base, J
64
65
66 def orth_projectors_from_J(J, rtol=1e-12):
67     U, s, VT = npl.svd(J, full_matrices=True)
68     r = int(np.sum(s > rtol * s[0]))
69     Ur = U[:, :r]
70     Us = U[:, r:]
71     Pi_r = Ur @ Ur.T
72     Pi_s = Us @ Us.T
73     return {"U": U, "s": s, "VT": VT, "r": r, "Ur": Ur, "Us": Us,
74           "Pi_r": Pi_r, "Pi_s": Pi_s}
75
76
77 def eigspace_overlap(A, B, k=3):
78     wa, Va = npl.eigh(A)
79     wb, Vb = npl.eigh(B)
80     Qa = Va[:, np.argsort(wa)[::-1][:k]]
81     Qb = Vb[:, np.argsort(wb)[::-1][:k]]
82     M = Qa.T @ Qb
83     s = npl.svd(M, compute_uv=False)
84     return float(np.mean(s**2))
85
86
87 def relative_fro_error(A, B):
88     return float(
89         npl.norm(A - B, ord="fro")
90         / (npl.norm(B, ord="fro") + 1e-30)
91     )
92
93
94 def parameter_rmse(T_hat, theta_true):
95     E = T_hat - theta_true[None, :]
96     return np.sqrt(np.mean(E**2, axis=0))
97
98
99 def parameter_std(T_hat):
100     return np.std(T_hat, axis=0, ddof=1)
101
102
103 def principal_overlap_with_null(C, Us, k=3):
104     w, V = npl.eigh(C)
105     Q = V[:, np.argsort(w)[::-1][:k]]
106     k_eff = min(k, Us.shape[1])
107     if k_eff == 0:
108         return np.nan
109     U0 = Us[:, :k_eff]
110     M = Q.T @ U0
111     s = npl.svd(M, compute_uv=False)

```

```

112     return float(np.mean(s[:k_eff]**2))
113
114
115 def fisher(J, C):
116     Ci = npl.pinv(C)
117     return J.T @ Ci @ J
118
119
120 def gls_update(theta0, y, base, J, C, precondition=False, ridge=1e-12):
121     col_scale = np.ones(J.shape[1])
122     J_use = J.copy()
123     if precondition:
124         col_norms = np.linalg.norm(J, axis=0)
125         col_scale = np.where(col_norms > 0, col_norms, 1.0)
126         J_use = J / col_scale[None, :]
127     Ci = npl.pinv(C)
128     A = J_use.T @ Ci @ J_use
129     b = J_use.T @ Ci @ (y - base)
130     reg = ridge * (np.trace(A) if np.isfinite(np.trace(A)) else 1.0)
131     dz = npl.pinv(A + reg * np.eye(A.shape[0])) @ b
132     dtheta = dz / col_scale
133     theta_hat = theta0 + dtheta
134     F = J_use.T @ Ci @ J_use
135     evals = npl.eigvalsh(F)
136     lam_max = float(np.max(evals)) if evals.size else 0.0
137     pos = evals[evals > 0]
138     kappa = float(lam_max / np.min(pos)) if pos.size else np.inf
139     return theta_hat, {"F": F, "kappa": kappa, "evals": evals}
140
141
142 def canonical_covariance_estimator(residuals, Pi_s, Sigma_eps,
143                                   lam=0.25, rho=1e-12):
144     R = residuals - residuals.mean(axis=0, keepdims=True)
145     Shat = (R.T @ R) / max(len(residuals) - 1, 1)
146     Sproj = Pi_s @ Shat @ Pi_s
147     w, V = npl.eigh(Pi_s)
148     U_s = V[:, w > 0.5]
149     Ss = U_s.T @ Sproj @ U_s
150     Lambda = ((1.0 - lam) * Ss
151               + lam * np.diag(np.diag(Ss))
152               + rho * np.eye(Ss.shape[0]))
153     Cseep_hat = U_s @ Lambda @ U_s.T
154     Ceff_hat = Cseep_hat + Sigma_eps
155     return {"Shat": Shat, "Sproj": Sproj, "U_s": U_s,
156           "Lambda_hat": Lambda, "Cseep_hat": Cseep_hat,
157           "Ceff_hat": Ceff_hat}
158
159
160 def make_sensor_noise_cov(m, sigma_min=1e-8, sigma_max=4e-8, seed=0):
161     sig = np.linspace(sigma_min, sigma_max, m)
162     C = np.diag(sig**2)
163     for i in range(m - 1):
164         C[i, i+1] = C[i+1, i] = 0.15 * sig[i] * sig[i+1]
165     return C
166
167
168 def sample_heavy_tailed_noise(rng, Sigma, p_burst=0.08,
169                               burst_scale=10.0):

```



```

169     z = rng.multivariate_normal(np.zeros(Sigma.shape[0]), Sigma)
170     if rng.random() < p_burst:
171         z = z + burst_scale * rng.multivariate_normal(
172             np.zeros(Sigma.shape[0]), Sigma
173         )
174     return z
175
176
177 def leakage_ratio(Cseep_hat, Pi_r, Pi_s):
178     num = (npl.norm(Pi_r @ Cseep_hat @ Pi_r, ord="fro")
179            + npl.norm(Pi_r @ Cseep_hat @ Pi_s, ord="fro"))
180     den = npl.norm(Cseep_hat, ord="fro") + 1e-30
181     return float(num / den)
182
183
184 def run_experiment(W=80, seed=7, precondition=True):
185     rng = np.random.default_rng(seed)
186
187     conductors = [loop_polyline(0.05, 400)]
188     sensors = axial_and_radial_sensors(nz=60, zmax=0.06,
189                                       nr=40, rmax=0.08)
190
191     m = sensors.shape[0]
192
193     theta_true = np.array([4.0, 1.2e-3, -7.5e-4], dtype=float)
194     theta0 = np.array([4.0, 0.0, 0.0], dtype=float)
195
196     base0, J0 = jacobian_fd(theta0, sensors, conductors)
197     proj = orth_projectors_from_J(J0)
198     Pi_r, Pi_s, U_s = proj["Pi_r"], proj["Pi_s"], proj["Us"]
199
200     q = U_s.shape[1]
201     raw_spec = np.geomspace(4e-15, 2e-10, max(q, 1))
202     raw_spec[:min(3, q)] = np.array([8e-11, 3e-11, 1.5e-11])[:min(3,
203     q)]
204     raw_spec[min(3, q):] *= 0.15
205     Lambda_true = np.diag(raw_spec) if q > 0 else np.zeros((0, 0))
206     Cseep_true = U_s @ Lambda_true @ U_s.T if q > 0 else np.zeros((m,
207     m))
208
209     Sigma_eps = make_sensor_noise_cov(m, seed=seed + 100)
210
211     Y = []
212     for _ in range(W):
213         y_clean = forward_Bz(theta_true, sensors, conductors)
214         u = (rng.multivariate_normal(np.zeros(m), Cseep_true)
215             if q > 0 else np.zeros(m))
216         eps = sample_heavy_tailed_noise(rng, Sigma_eps)
217         Y.append(y_clean + u + eps)
218     Y = np.asarray(Y)
219
220     sigma2_id = float(np.median(np.diag(Sigma_eps)))
221     C_id = sigma2_id * np.eye(m)
222
223     theta_hat_I, resid_I = [], []
224     for y in Y:
225         th, _ = gls_update(theta0, y, base0, J0, C_id,
226                           precondition=precondition)
227         theta_hat_I.append(th)

```

```

225     resid_I.append(y - forward_Bz(th, sensors, conductors))
226     theta_hat_I = np.asarray(theta_hat_I)
227     resid_I = np.asarray(resid_I)
228
229     est = canonical_covariance_estimator(resid_I, Pi_s, Sigma_eps,
230                                         lam=0.35, rho=1e-15)
231     Ceff_hat = est["Ceff_hat"]
232
233     theta_hat_C = []
234     for y in Y:
235         th, _ = gls_update(theta0, y, base0, J0, Ceff_hat,
236                           precondition=precondition)
237         theta_hat_C.append(th)
238     theta_hat_C = np.asarray(theta_hat_C)
239
240     rmse_I = parameter_rmse(theta_hat_I, theta_true)
241     rmse_C = parameter_rmse(theta_hat_C, theta_true)
242     std_I = parameter_std(theta_hat_I)
243     std_C = parameter_std(theta_hat_C)
244
245     Jn = J0 / np.linalg.norm(J0, axis=0, keepdims=True)
246     FI = fisher(Jn, C_id) if precondition else fisher(J0, C_id)
247     FC = fisher(Jn, Ceff_hat) if precondition else fisher(J0, Ceff_hat)
248     wI = npl.eigvalsh(FI)
249     wC = npl.eigvalsh(FC)
250     kappa_I = float(np.max(wI) / np.min(wI[wI > 0]))
251     kappa_C = float(np.max(wC) / np.min(wC[wC > 0]))
252
253     overlap_top3 = eigspace_overlap(est["Cseep_hat"], Cseep_true,
254                                     k=min(3, m))
255     null_overlap = principal_overlap_with_null(est["Cseep_hat"], U_s,
256                                                k=min(3, U_s.shape[1]))
257     fro_err = relative_fro_error(est["Cseep_hat"], Cseep_true)
258     leak = leakage_ratio(est["Cseep_hat"], Pi_r, Pi_s)
259
260     C_exact_from_truth = Pi_s @ Cseep_true @ Pi_s
261     exact_uniqueness_error = relative_fro_error(C_exact_from_truth,
262                                                Cseep_true)
263
264     rows = []
265     for i in range(0, m - 1, 2):
266         row = np.zeros(m)
267         row[i] = 0.5
268         row[i+1] = 0.5
269         rows.append(row)
270     if m % 2 == 1:
271         row = np.zeros(m)
272         row[-1] = 1.0
273         rows.append(row)
274     P = np.vstack(rows)
275
276     Jcg = P @ J0
277     Ccg = P @ Ceff_hat @ P.T
278     F_full = fisher(J0, Ceff_hat)
279     F_cg = fisher(Jcg, Ccg)
280     dif = F_full - F_cg
281     eig_dif = npl.eigvalsh((dif + dif.T) * 0.5)
282     cg_pass = bool(np.min(eig_dif) >= -1e-8)

```

```

283         * np.max(np.abs(eig_dif) + 1e-30))
284
285     return {
286         "theta_true": theta_true.tolist(),
287         "rmse_identity": rmse_I.tolist(),
288         "rmse_canonical": rmse_C.tolist(),
289         "std_identity": std_I.tolist(),
290         "std_canonical": std_C.tolist(),
291         "kappa_identity_preconditioned": kappa_I,
292         "kappa_canonical_preconditioned": kappa_C,
293         "fro_error_Cseep": fro_err,
294         "top3_eigenspace_overlap": overlap_top3,
295         "null_space_overlap": null_overlap,
296         "leakage_ratio": leak,
297         "exact_uniqueness_error": exact_uniqueness_error,
298         "coarse_graining_pass": cg_pass,
299         "dim_null": int(U_s.shape[1]),
300         "windows": int(W),
301     }
302
303
304 if __name__ == "__main__":
305     out = run_experiment(W=80, seed=7, precondition=True)
306     for k, v in out.items():
307         print(f"{k}: {v}")

```

7 What is proved, what is tested, and what is not claimed

The synthetic battery separates three logically distinct claims.

Support uniqueness. The admissible support projector Π_s is fixed uniquely by the Jacobian. This is a theorem-level statement, independent of simulation.

Exact covariance uniqueness. If C_r and Σ_ε are known exactly, then [theorem 2](#) identifies a unique admissible seepage covariance. This is the strongest mathematically valid form of the induced-covariance claim.

Finite-sample recoverability. The numerical test concerns only whether the projected residual estimator recovers a good approximation to the true null-supported covariance and thereby improves inversion while respecting admissibility gates.

Limitations and non-claims

The present note does *not* claim any of the following:

- (a) that the Jacobian alone determines the full unresolved covariance spectrum;
- (b) that every latent effect in every inverse problem must satisfy $J^\top u = 0$;
- (c) that finite-sample projected covariance recovery is guaranteed under strong nonlinearity, severe window mismatch, or insufficient data;
- (d) that improved numerical fit alone proves the orthogonal-seepage hypothesis.

Instead, the claim is limited to this: *under the CTMT admissibility hypothesis on stabilized windows, the Jacobian uniquely determines the support projector of unresolved covariance, and exact covariance uniqueness follows once residual second moments and sensor noise are fixed.*

8 Suggested results framing

A reviewer-resistant description of the supporting numerical outcome is the following.

In the synthetic coil test, the Jacobian-null support projector was fixed uniquely by the local linearization. When the true latent seepage covariance was generated inside this support, the projected residual estimator recovered its dominant structure with substantial eigenspace overlap and low leakage into resolved directions. Relative to identity weighting, the resulting effective covariance reduced parameter RMSE and markedly reduced window-to-window estimator spread. Coarse-graining monotonicity was preserved in accepted runs. These results support a restricted induced-covariance claim: the Jacobian fixes the admissible unresolved support, while residual data identify the unresolved spectrum within that support.

9 Falsifier and failure modes

The closure principle is contradicted if any of the following occurs on stabilized windows:

- (1) **Support leakage:** persistent large ℓ in (23), indicating unresolved covariance outside $\ker(J^\top)$;
- (2) **Cross-block inconsistency:** failure of (8);
- (3) **No inversion gain:** projected covariance does not reduce RMSE or estimator spread relative to the identity baseline;
- (4) **Monotonicity violation:** coarse-graining yields $F_{cg} \not\leq F$.

These falsifiers are stronger and more informative than merely observing or failing to observe improved fit.

10 Discussion

The central point is not that residuals have covariance; that is unavoidable. The stronger claim is that under the stated CTMT admissibility postulate, unresolved covariance is not free to occupy arbitrary observable directions. Its admissible support is constrained by the Jacobian:

This makes seepage covariance geometrically anchored. The logical chain is therefore rather than

Within the limits of the synthetic experiment, covariance appears as an induced object rather than as an externally imposed prior.

References

- [1] N. N. Cencov. *Statistical Decision Rules and Optimal Inference*. American Mathematical Society, 1982.
- [2] Shun-ichi Amari and Hiroshi Nagaoka. *Methods of Information Geometry*. AMS and Oxford University Press, 2000.
- [3] C. R. Rao. *Linear Statistical Inference and Its Applications*. Wiley, 1973.
- [4] Adi Ben-Israel and Thomas N. E. Greville. *Generalized Inverses: Theory and Applications*. Springer, 2003.
- [5] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2013.

1

¹A useful way to interpret the CTMT admissibility postulate is to view it as formalizing a basic structural feature of measurement itself. Any physical measurement involves a distinction between the moment when a physical event occurs and the moment when information about that event becomes available to the observer. The delay between these two moments is not merely a nuisance but a geometric constraint: only certain directions in observable space can affect the resolved parameters, while others remain intrinsically unobservable. In this sense, unresolved effects may be regarded as “leaking” into directions that do not carry resolved sensitivity. The Jacobian identifies these directions precisely as the Fisher-null subspace $\ker(J^\top)$, which is the unique set of perturbations that can vary without mimicking parameter motion. Thus the admissibility condition $J^\top u = 0$ is not an arbitrary modeling choice but the structural requirement that unresolved transport must not interfere with resolved inference. Once this geometric constraint is imposed, the unresolved covariance is forced to live entirely within the Jacobian-null support, and the combination of this support with exact residual second moments yields the uniqueness result proved in the text.